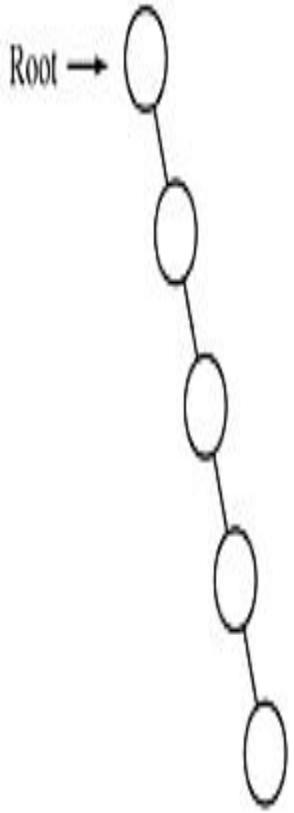
AVL Tree

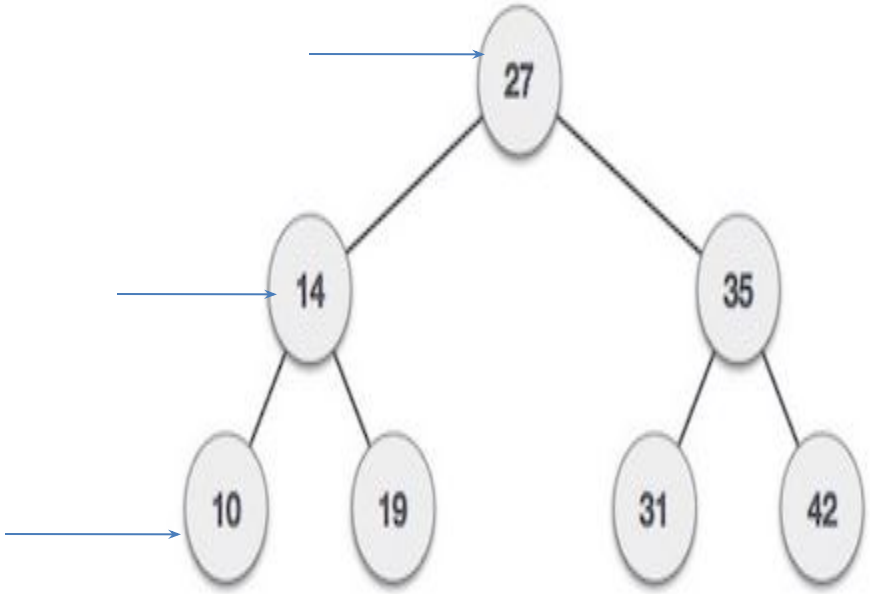
Problem with Binary search tree

- The disadvantage of a binary search tree is that its height can be as large as N-1
- This means that the time needed to perform insertion and deletion and many other operations can be N times in the worst case
- We want a tree with small height
- A binary tree with N node has height at least $\log_2 (N+1)-1$
- Thus, our goal is to keep the height of a binary search tree $\log_2 (N)$ times
- Such trees are called balanced binary search trees. Examples are AVL tree, red-black tree.

Binary search tree



Unbalanced Tree



balanced Tree

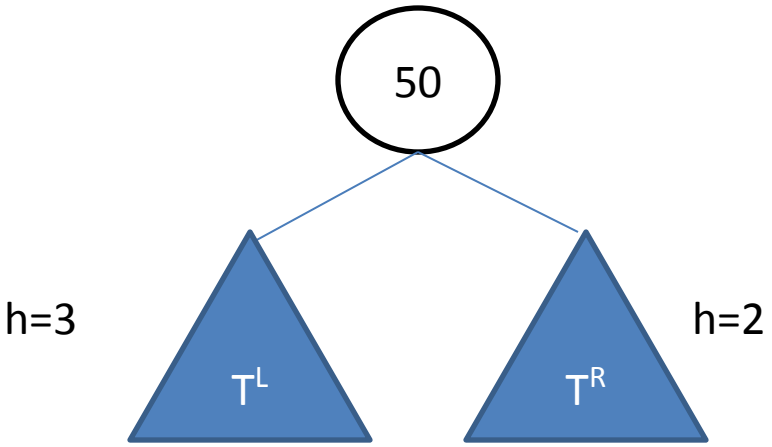
AVL

- There is a need to maintain the binary search tree to be of balanced height, so that it is possible to obtain for the search option of $\log_2$ (N) time in the worst case.
- One of the most popular balanced tree was introduced by Adelson velskii and landis (AVL)

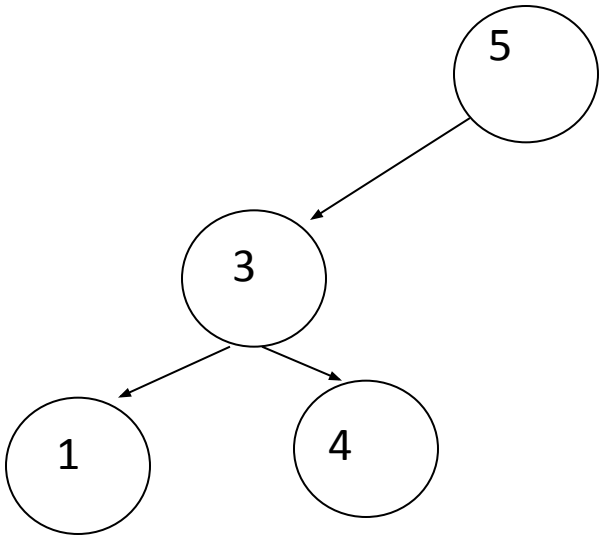
AVL Tree

- An empty binary search tree is an AVL tree
- A non empty binary tree T is an AVL tree iff T^L (left subtree) and T^R (right subtree) of T and $h(T^L)$ (height of left subtree) and $h(T^R)$ (height of right subtree) where $|h(T^L) - h(T^R)| <= 1$
- Balance factor BF is difference between $h(T^L)$ and $h(T^R)$ and the value will be -1 0 1

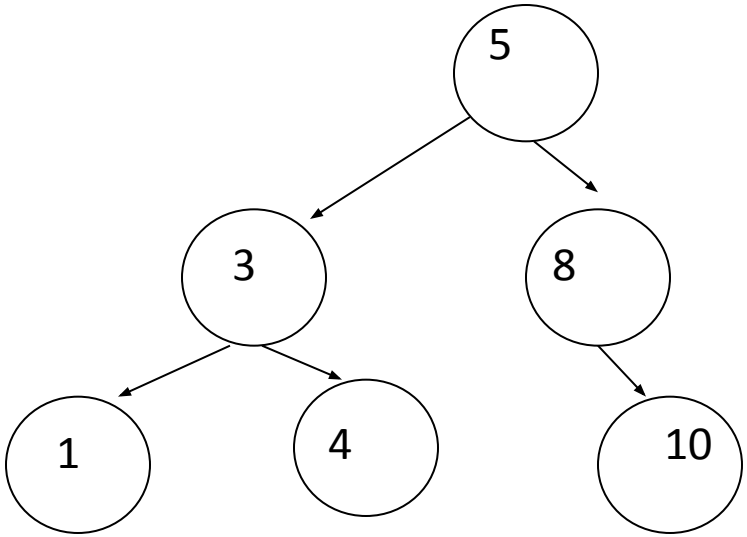
$$BF = h(T^L) - h(T^R)$$



BF = 2-1 =1



Not AVL Tree



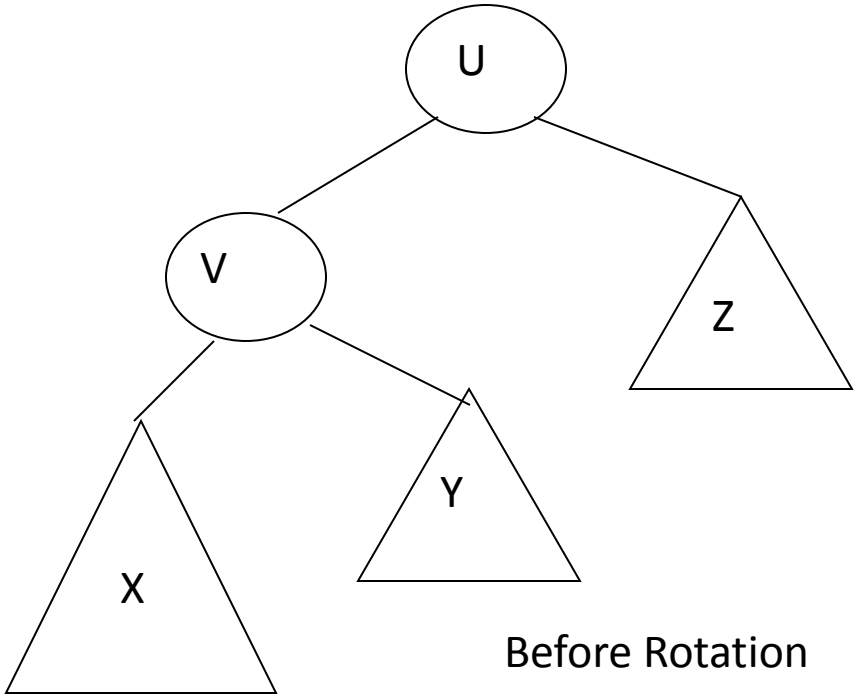
AVL Tree

Rotations

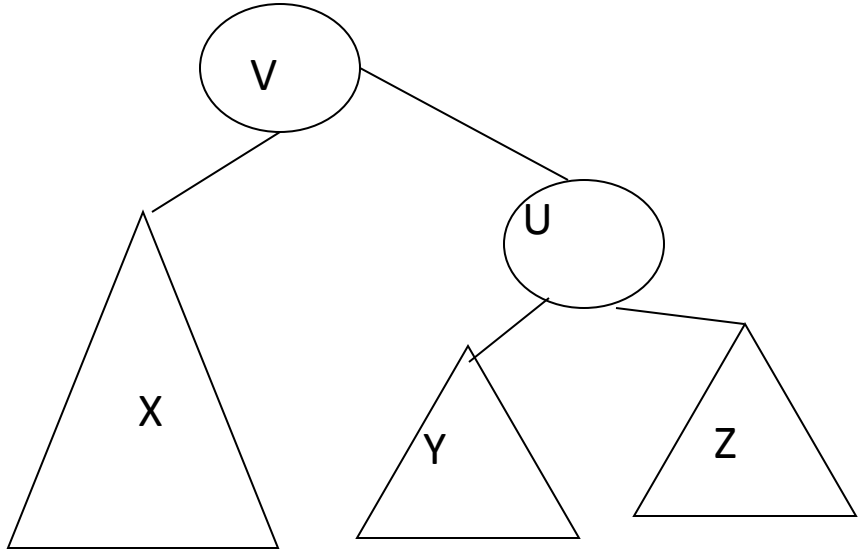
- Left to left: Inserted node is in the left subtree of left subtree of node A
- Right to right: Inserted node is in the right subtree of right subtree of node A
- Right to left: Inserted node is in the right subtree of left subtree of node A
- Left of right: Inserted node is in the left subtree of right subtree of node A
- **Trick to Solve**
  1. For LL and RR find A and B and make A as child of B
  2. For LR and RL find A , B and C and make A, B as child of C

Insertion in left child of left subtree

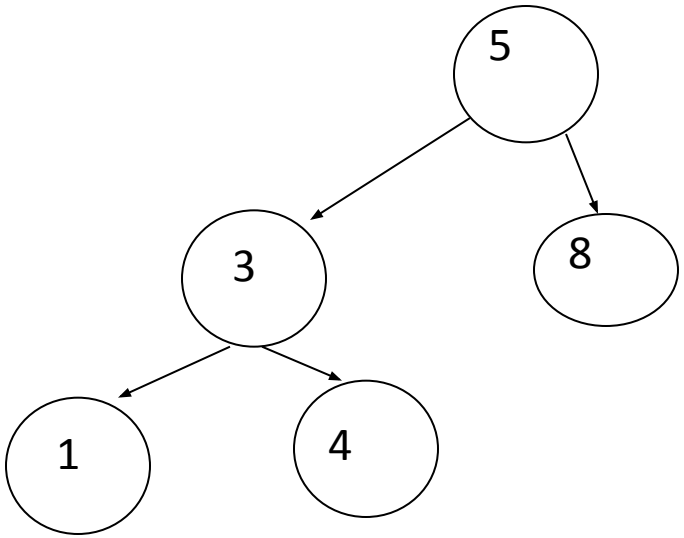
Single Rotation



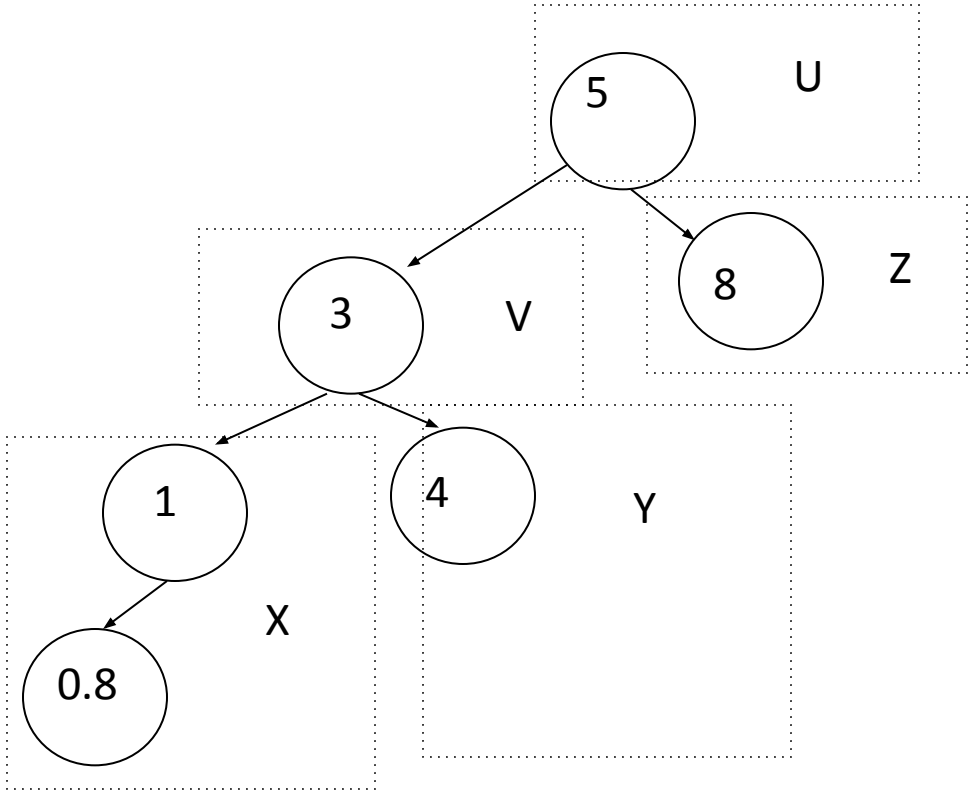
Before Rotation



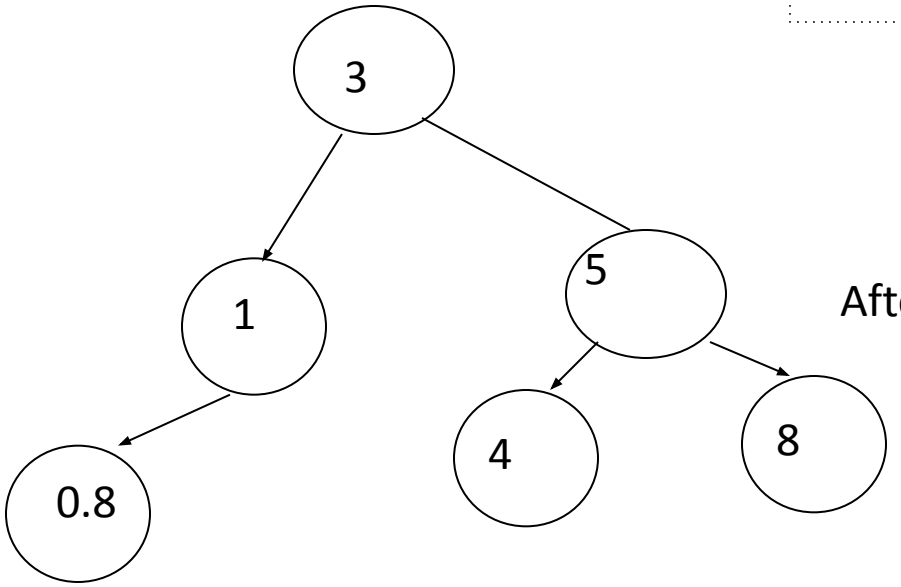
After Rotation



AVL Tree



Insert 0.8

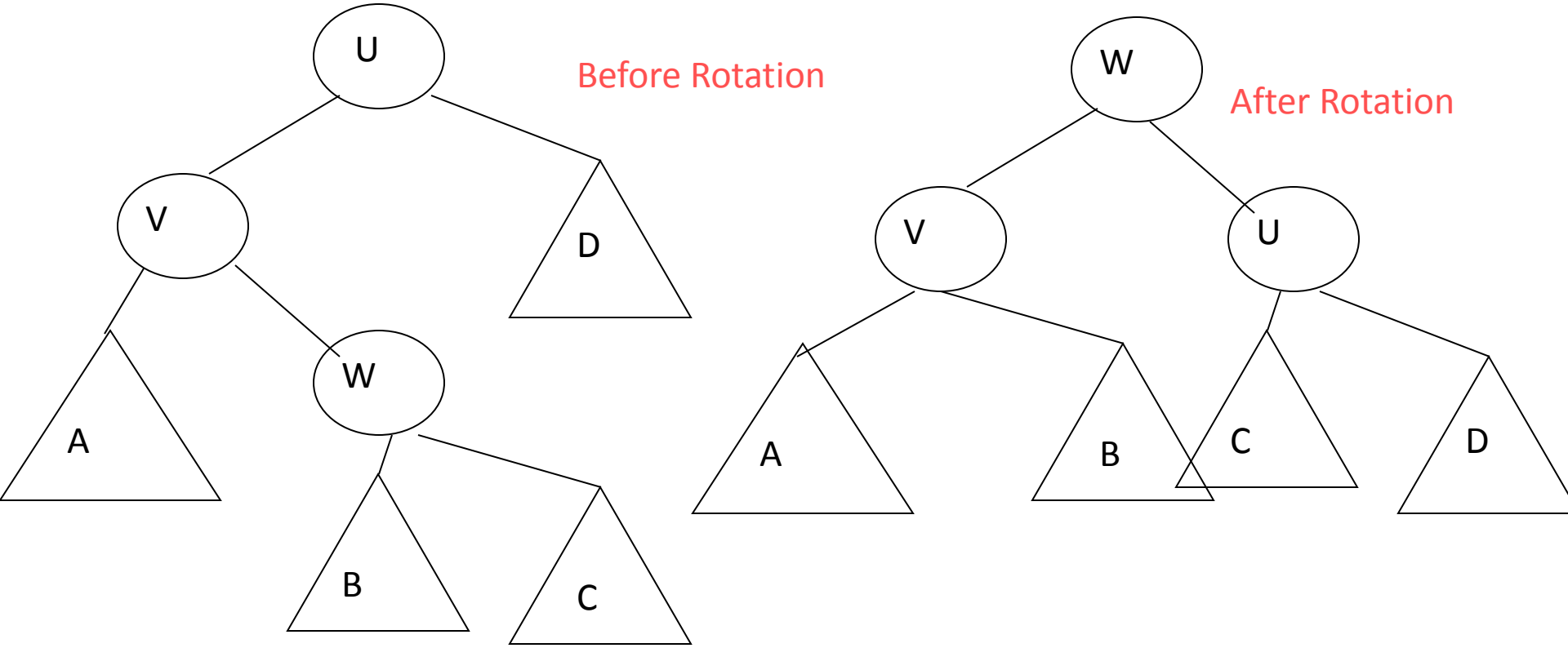


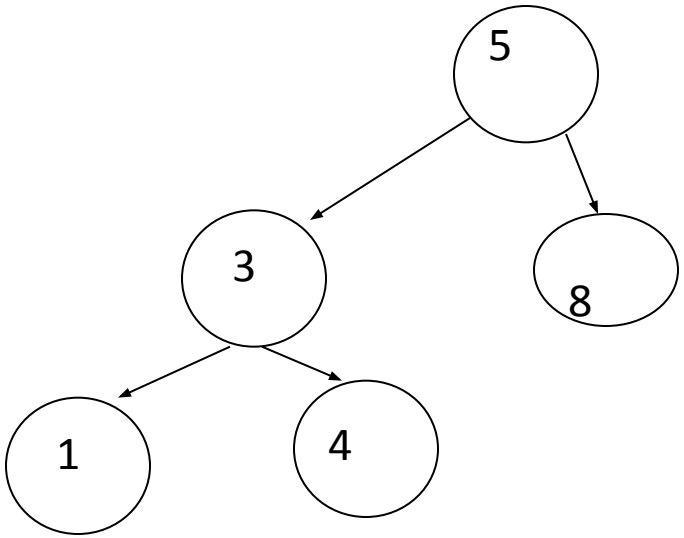
After Rotation

Double Rotation

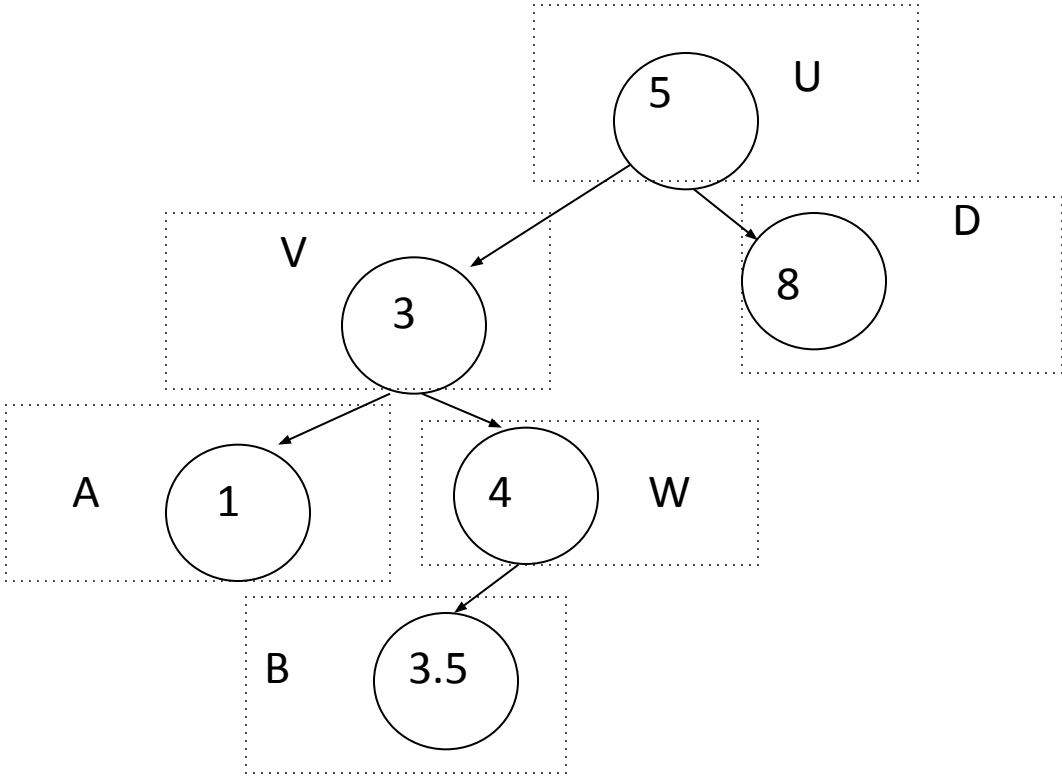
Suppose, imbalance is due to an insertion in the left subtree of right child

Single Rotation does not work!

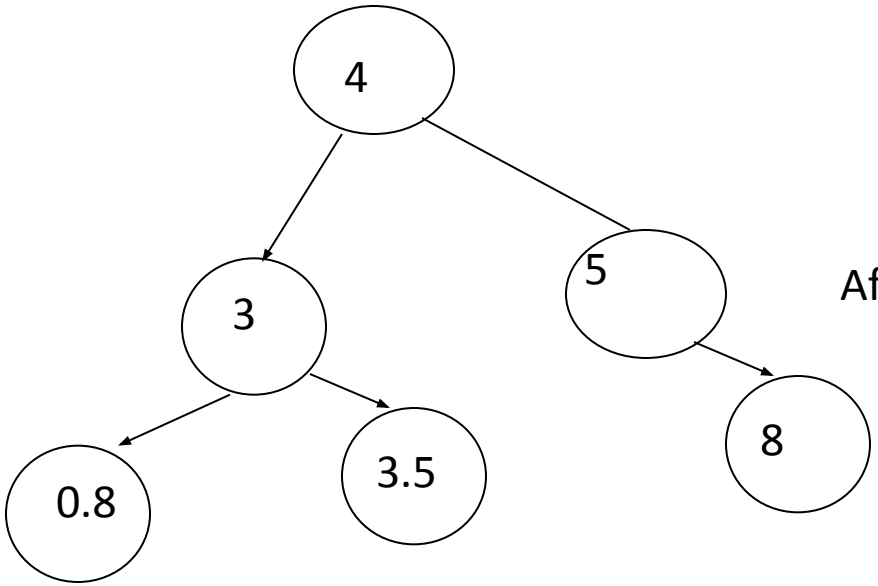




AVL Tree



Insert 3.5



After Rotation

Extended Example

Insert 3,2,1,4,5,6,7, 16,15,14

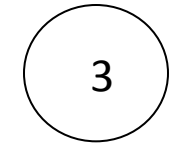


Fig 1

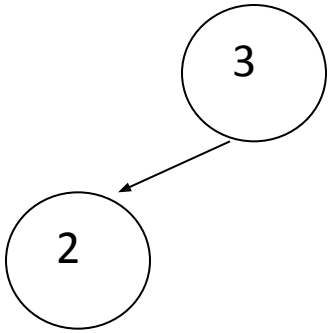


Fig 2

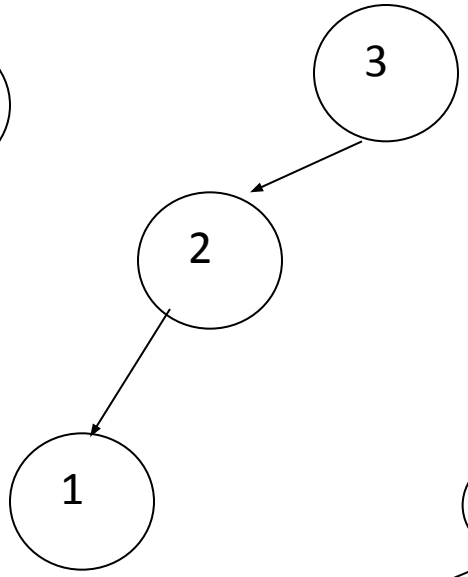


Fig 3

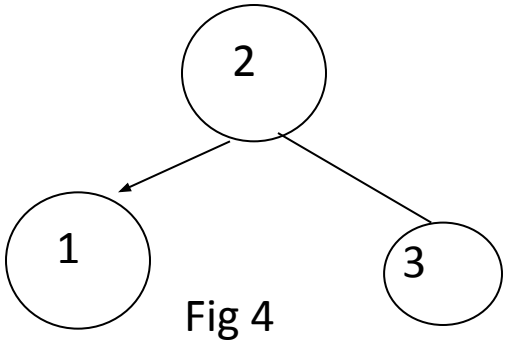


Fig 4

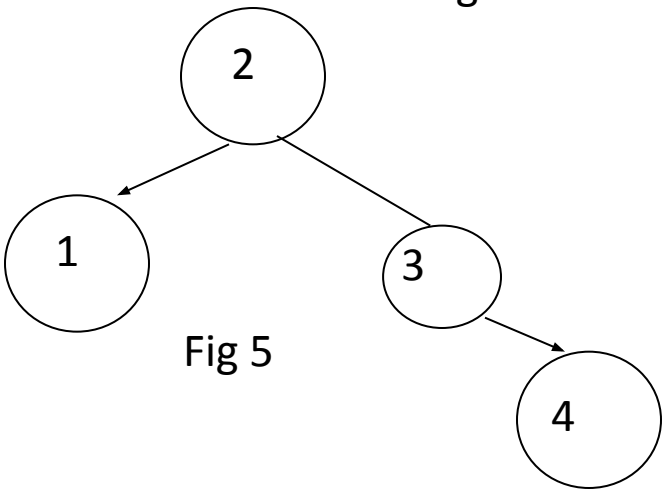


Fig 5

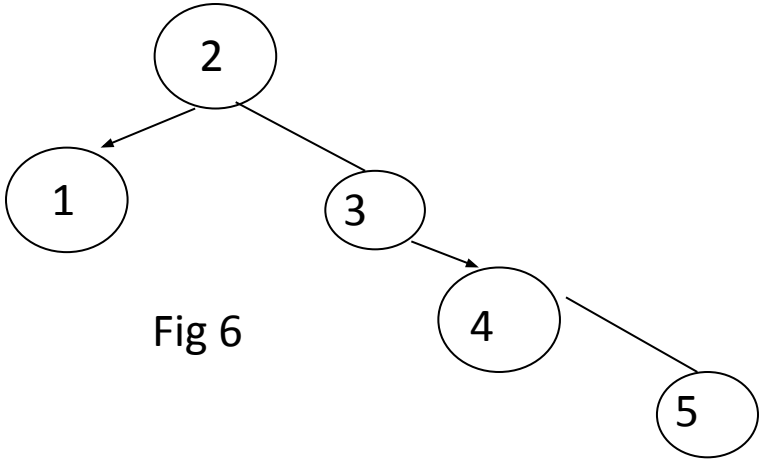


Fig 6

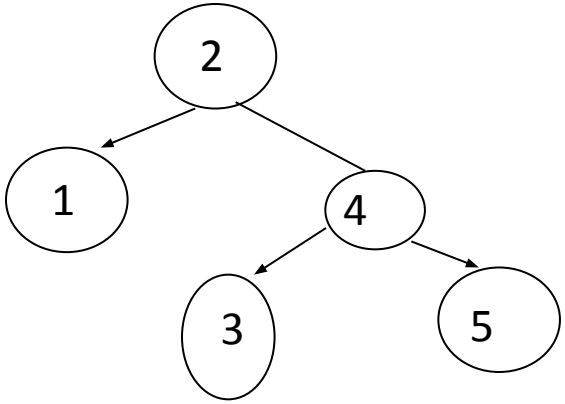


Fig 7

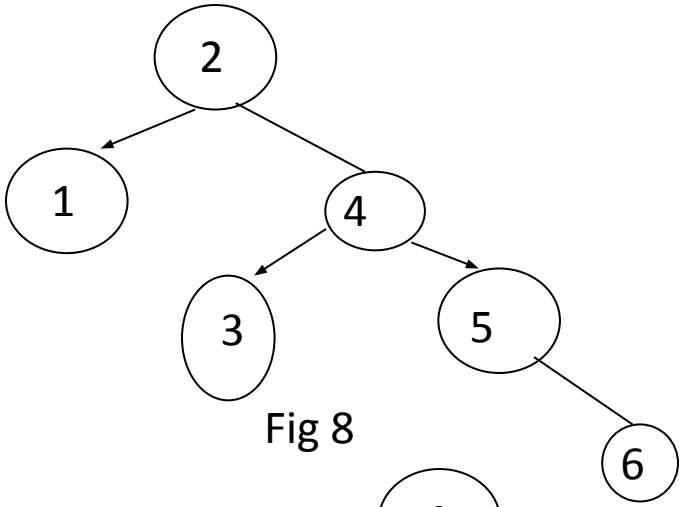


Fig 8

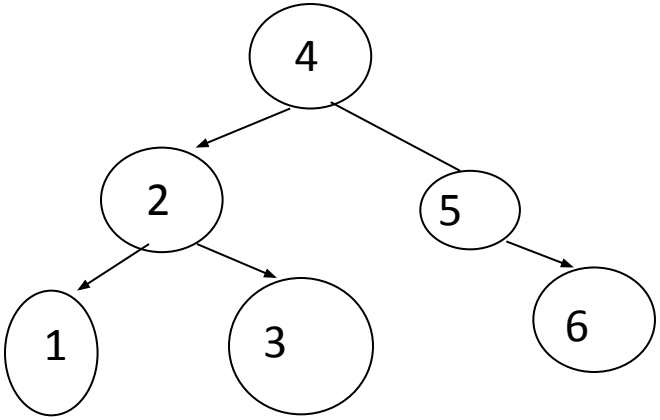


Fig 9

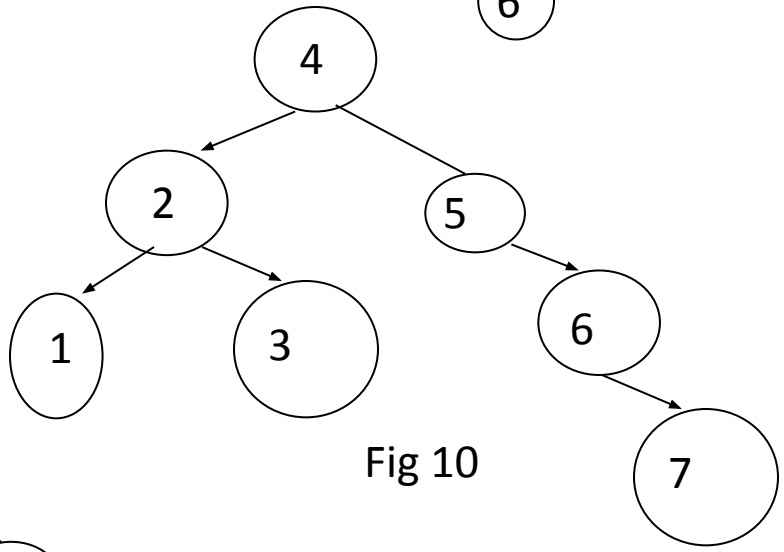


Fig 10

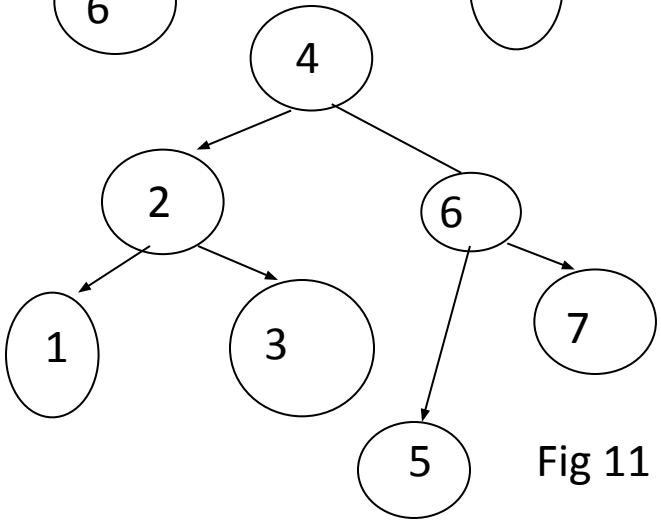


Fig 11

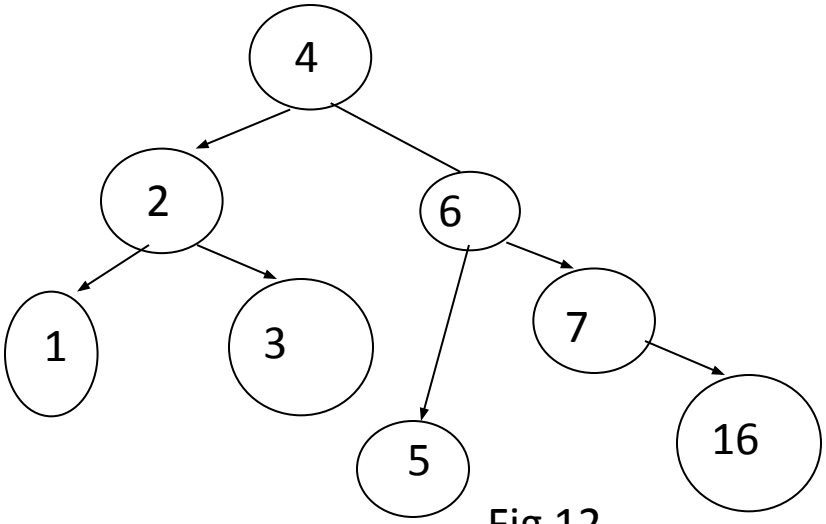


Fig 12

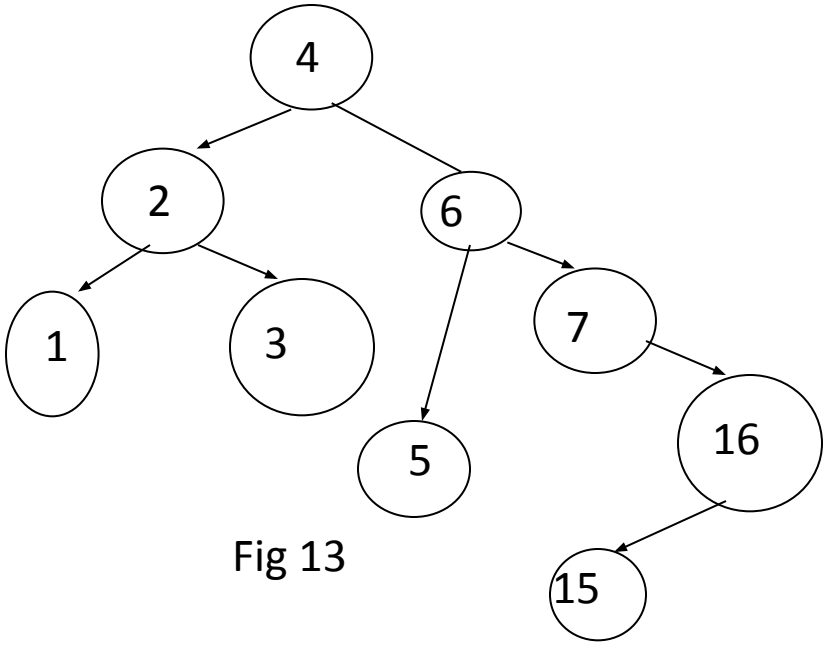


Fig 13

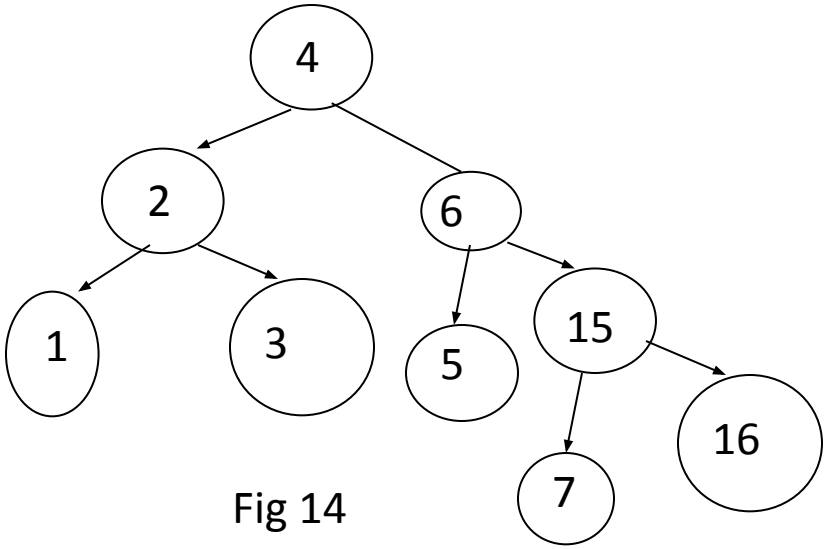


Fig 14

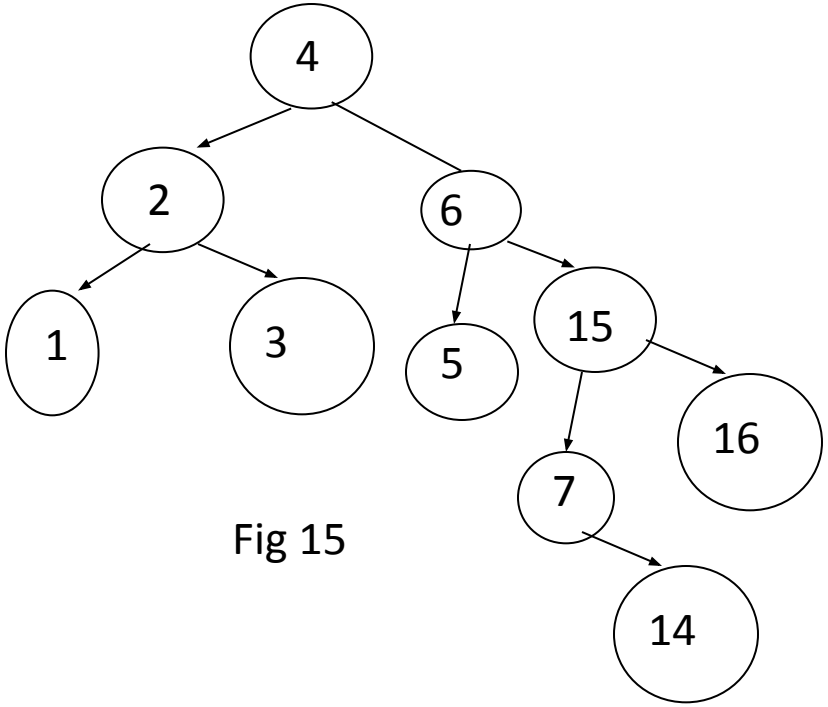


Fig 15

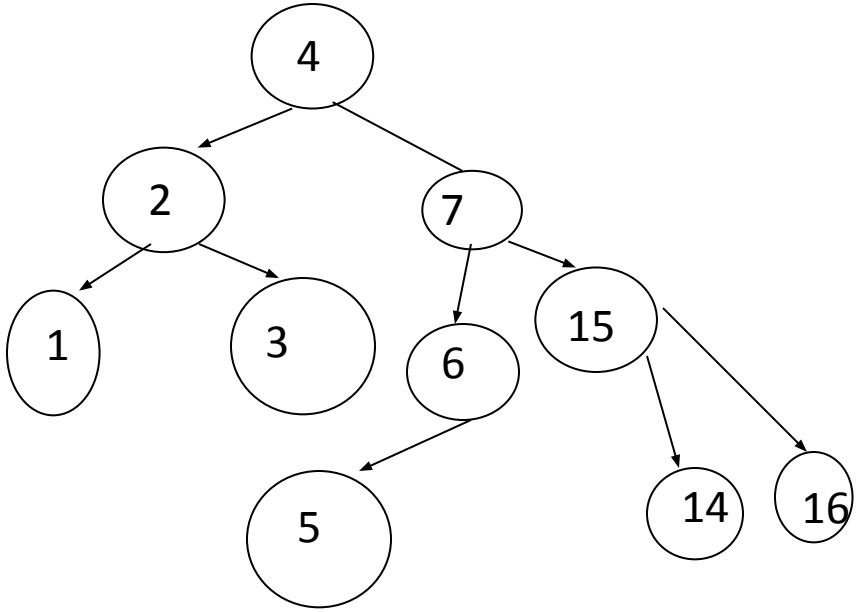
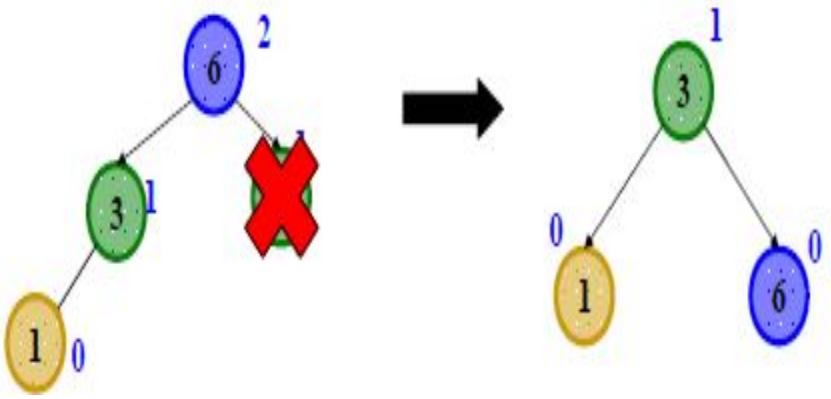


Fig 16

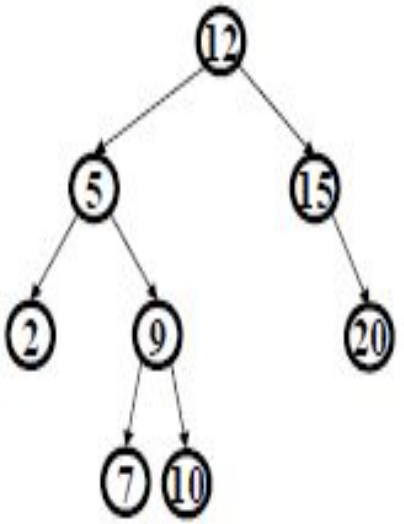
AVL Tree Deletion

- Similar to insertion: do the delete and then rebalance
  - Rotations and double rotations
  - Imbalance may propagate upward so rotations at multiple nodes along path to root may be needed (unlike with insert)
- Simple example: a deletion on the right causes the left-left grandchild to be too tall



Properties of BST delete

We first do the normal BST deletion:
– 0 children: just delete it
– 1 child: delete it, connect child to parent
– 2 children: put successor in your place, delete successor leaf

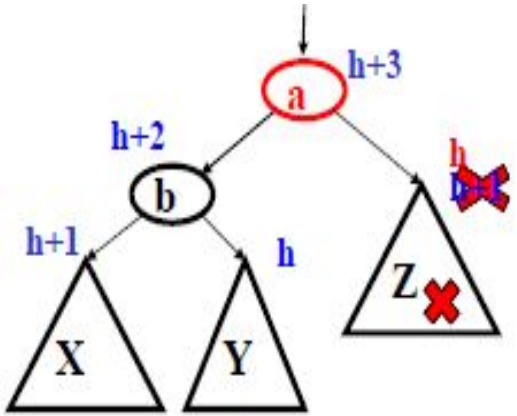


Which nodes' heights may have changed:
– 0 children: path from deleted node to root
– 1 child: path from deleted node to root
– 2 children: path from *deleted successor leaf to root*

Will rebalance as we return along the "path in question" to the root

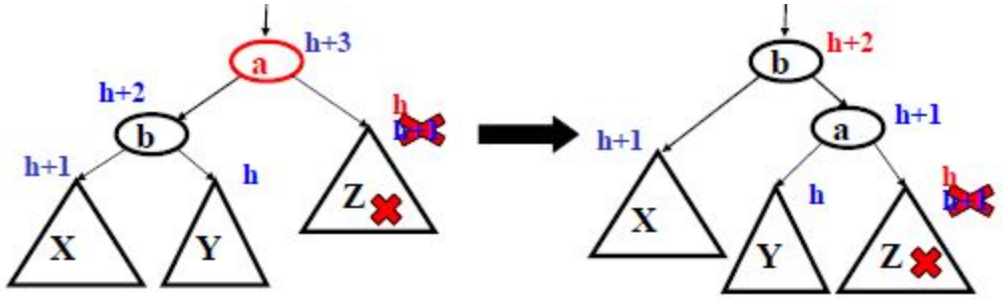
Case #1 Left-left due to right deletion

- Start with some subtree where if right child becomes shorter we are unbalanced due to height of left-left grandchild



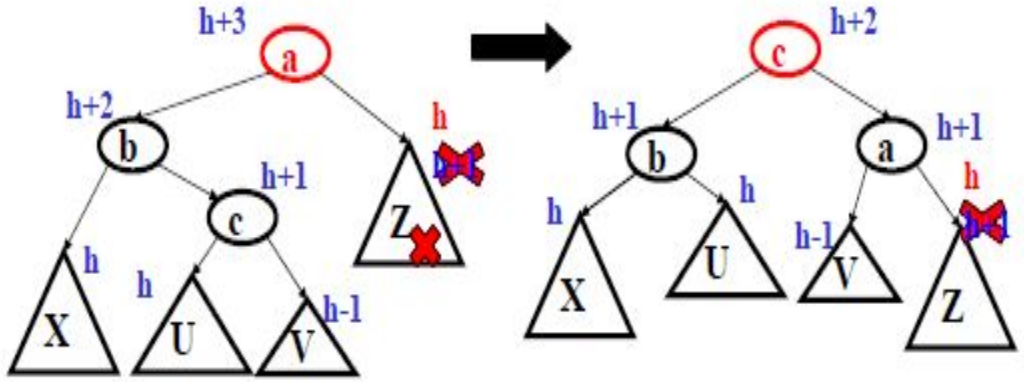
- A delete in the right child could cause this right-side shortening

Case #1: Left-left due to right deletion



- Same single rotation as when an insert in the left-left grandchild caused imbalance due to X becoming taller
  - But here the "height" at the top decreases, so more rebalancing farther up the tree might still be necessary

Case #2: Left-right due to right deletion



- Same double rotation when an insert in the left-right grandchild caused imbalance due to c becoming taller
- But here the "height" at the top decreases, so more rebalancing farther up the tree might still be necessary

AVL Tree Deletion

Naturally there are two mirror-image cases not shown here

– Deletion in left causes right-right grandchild to be too tall
– Deletion in left causes right-left grandchild to be too tall